

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I A G Jayapradhan (192471016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A G Jayapradhan

Reverse Engineering

1. Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using **safe, non-executable datasets or publicly available research samples**.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.
- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware analysis.

4. Reverse Engineering of APIs Using AI

Description:

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.
- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery.

Annexure A: Reverse Engineering - Comprehensive Task Solutions

Task 1: Machine Learning Techniques For Reverse Engineering

1. System/Conceptual Overview & Objectives

Reverse engineering of software components often involves analyzing large volumes of compiled or obfuscated code to understand underlying functionalities. The integration of Machine Learning (ML) into this domain shifts the paradigm from manual, heuristic-based inspection to automated, data-driven analysis.

The primary objective of this task is to utilize ML algorithms to classify software components, identify recurring programming patterns, detect anomalous behaviors, and predict the functionality of unknown code segments.

By leveraging supervised, unsupervised, and semi-supervised learning approaches, analysts can construct predictive models capable of understanding code semantics even when traditional documentation and source code are absent.

2. Technical Methodology & AI Approaches

The methodology relies heavily on robust feature engineering. Before ML algorithms can process software, raw code (often binaries or assembly) must be transformed into vector representations.

Common techniques include generating Abstract Syntax Trees (ASTs) or Control Flow Graphs (CFGs). From these structures, features such as opcode n-grams, cyclomatic complexity, and API call frequencies are extracted.

For supervised learning, algorithms like Random Forest or Support Vector Machines (SVM) are trained on labeled datasets (e.g., cryptographic functions vs. network I/O functions). Unsupervised learning, utilizing K-Means or DBSCAN, is critical for clustering 'code clones' or identifying similar structural patterns across disparate binaries without prior labeling.

3. Detailed Student Activity Implementation

Phase 1 (Data Collection): Students will compile a diverse dataset of source-code samples or decompiled binaries across various languages (C++, Python, Java) from platforms like GitHub.

Phase 2 (Feature Extraction): Using tools like Joern or Radare2, students will extract structural and syntactical features, converting the code base into a high-dimensional feature matrix.

Phase 3 (Algorithm Application): Students will apply classification algorithms to categorize the code by functionality and clustering algorithms to group similar programs. This requires splitting the dataset into training, validation, and testing sets.

Phase 4 (Analysis): The final step involves a deep comparative analysis. Students must map the algorithmic predictions back to the actual code to verify if the ML model accurately captured the semantic intent of the program.

4. Evaluation Metrics & Analytical Framework

Evaluation of supervised models will rely on standard classification metrics: Precision, Recall, and the F1-Score. Given that code datasets are often imbalanced (e.g., many UI functions but few cryptographic ones), the F1-Score provides a more reliable metric than raw accuracy.

For unsupervised clustering of code similarities, metrics such as the Silhouette Score will be used to determine the density and separation of the identified code clusters.

Performance benchmarking will also factor in the computational overhead of the feature extraction phase, as highly complex CFGs can cause exponentially increasing processing times.

5. Expected Outcomes & Real-World Impact

The expected outcome is a comprehensive evaluation of different ML algorithms' suitability for software reverse-engineering tasks.

Students will gain a profound understanding of the limitations of ML in this space—specifically, how obfuscation techniques (like variable renaming or dead code insertion) can easily deceive baseline ML models.

Ultimately, students will demonstrate that while ML cannot entirely replace human analysts in reverse engineering, it acts as a powerful triage tool, drastically reducing the time required to categorize and comprehend large, unknown codebases.

Task 2: AI for Source Code Analysis and Program Understanding

1. System/Conceptual Overview & Objectives

Modern software projects are characterized by immense scale and complexity, often comprising millions of lines of code across numerous interrelated microservices.

This task focuses on the application of Artificial Intelligence—specifically Natural Language Processing (NLP) and Large Language Models (LLMs)—to automatically analyze massive source-code repositories.

The goal is to demystify complex, legacy, or poorly documented code by having AI autonomously identify critical functions, trace dependency graphs, and convert obscure programming syntax into clear, human-readable explanations.

2. Technical Methodology & AI Approaches

The technical approach utilizes specialized LLMs fine-tuned on code, such as CodeBERT, StarCoder, or OpenAI's Codex. These models treat source code as a structured language, capturing semantic relationships between variables and functions across multiple files.

Beyond deep learning, traditional static analysis techniques are hybridized with AI. Graph Neural Networks (GNNs) can be deployed over the repository's dependency graph to understand how data flows through the application.

Prompt engineering becomes a vital methodology here. The AI is directed via specific prompts to 'Explain the time complexity of this function' or 'Identify all downstream dependencies if this class is modified', turning the LLM into an interactive code-comprehension engine.

3. Detailed Student Activity Implementation

Phase 1 (Repository Selection): Students will fork a large, open-source repository (e.g., the Linux Kernel, Kubernetes, or a complex Apache project) that is sufficiently complex to warrant automated analysis.

Phase 2 (Tool Integration): Students will integrate AI-assisted analysis tools (like GitHub Copilot, SonarQube with AI extensions, or custom scripts using the OpenAI API) to scan the repository.

Phase 3 (Mapping and Extraction): The students will guide the AI to map out the core architecture, identifying entry points, critical classes, and complex algorithms hidden within the repository.

Phase 4 (Summarization and Verification): The AI will be tasked with generating natural-language documentation for undocumented functions. Crucially, students must manually review these AI-generated explanations against the actual source code to verify accuracy and identify hallucinations.

4. Evaluation Metrics & Analytical Framework

The effectiveness of the AI will be measured using both qualitative and quantitative frameworks. Quantitatively, the BLEU (Bilingual Evaluation Understudy) or ROUGE scores can be used if comparing AI summaries to existing ground-truth developer comments.

Qualitatively, students will employ a peer-review framework to assess the 'Readability' and 'Technical Accuracy' of the AI's explanations.

A critical evaluation metric is the False Confidence Rate: instances where the AI confidently provides an incorrect explanation of the code's behavior, which poses a severe risk in real-world software maintenance.

5. Expected Outcomes & Real-World Impact

The expected outcome is a critical assessment of the capabilities and limitations of AI in understanding unfamiliar source code.

Students will conclude that while AI excels at summarizing localized functions and identifying boilerplate patterns, it often struggles with broad, system-wide architectural context and highly domain-specific logic.

This task equips students with the critical thinking skills required to leverage AI as an assistant rather than an infallible oracle in software engineering and code auditing.

Task 3: AI-Based Malware Reverse Engineering and Analysis

1. System/Conceptual Overview & Objectives

Malware analysis is a high-stakes domain where security analysts must deconstruct malicious software to understand its behavior, origins, and potential impact.

Given the sheer volume of new malware variants generated daily (often via automated polymorphic engines), manual reverse engineering is no longer scalable.

This task explores how AI assists security researchers by automating the identification of suspicious code patterns, malicious API calls, and embedded strings, utilizing safe, conceptual workflows and non-executable datasets.

2. Technical Methodology & AI Approaches

The methodology divides into static and dynamic AI analysis. For static analysis, ML models examine the Portable Executable (PE) headers, string imports, and byte-level histograms without executing the code. Term Frequency-Inverse Document Frequency (TF-IDF) can be applied to imported API names to find anomalies.

For dynamic analysis (behavioral), malware is run in a secure sandbox (like Cuckoo Sandbox). The sequence of system calls, registry modifications, and network requests are captured as time-series data.

Deep Neural Networks (DNNs) or ensemble methods like XGBoost are then trained on these behavioral logs. Sequence models, such as LSTMs (Long Short-Term Memory networks), are particularly effective at classifying malware families based on the chronological sequence of their API calls.

3. Detailed Student Activity Implementation

Phase 1 (Safe Data Acquisition): Strict safety protocols dictate that students will not handle live malware. Instead, they will use publicly available, safe research datasets like the EMBER dataset or the Microsoft Malware Classification Challenge dataset (which provides hex dumps and assembly logs).

Phase 2 (Feature Engineering): Students will extract relevant static features from these datasets, such as section entropy, opcode frequencies, and byte-sequence n-grams.

Phase 3 (Classification Model Building): Students will build and train ML classification pipelines to distinguish between benign software and various malware families (e.g., Ransomware vs. Trojans).

Phase 4 (Pattern Analysis): After training, students will use feature importance techniques (like SHAP values) to analyze exactly *which* suspicious patterns or API combinations caused the AI to flag the sample as malware.

4. Evaluation Metrics & Analytical Framework

Given the catastrophic cost of False Negatives (allowing malware to pass as benign), the evaluation framework must heavily penalize missed detections. Therefore, Recall and the ROC-AUC (Receiver Operating Characteristic - Area Under Curve) are the primary metrics.

Students will also evaluate the model's resilience to adversarial evasion techniques, such as when malware authors pad files with benign strings to lower the overall file entropy and bypass ML detectors.

Performance must also be measured by inference speed, as real-time anti-virus engines require sub-millisecond classification times.

5. Expected Outcomes & Real-World Impact

The expected outcome is a deep, functional understanding of how AI supports automated malware analysis at scale.

Students will successfully evaluate classification accuracy and demonstrate the ability to trace AI decisions back to specific malicious code features.

Ultimately, students will recognize that AI is a critical force multiplier in cybersecurity, capable of parsing millions of samples to isolate the highly sophisticated, novel threats that require manual human reverse engineering.

Task 4: Reverse Engineering of APIs Using AI

1. System/Conceptual Overview & Objectives

In the modern interconnected software ecosystem, Application Programming Interfaces (APIs) serve as the vital connective tissue between services. However, developers frequently encounter APIs that lack documentation, possess deprecated specifications, or are entirely closed-source.

This task explores the use of AI to reverse-engineer these black-box APIs.

By analyzing network traffic (requests and responses), observed behaviors, and partial source code, AI can be leveraged to autonomously infer undocumented endpoints, map parameter dependencies, and reconstruct missing data formats.

2. Technical Methodology & AI Approaches

The technical approach involves network traffic analysis combined with sequence-to-sequence ML models. Network sniffers capture raw HTTP/HTTPS traffic, generating large datasets of JSON/XML payloads.

AI models, particularly those based on the Transformer architecture, analyze this traffic. The AI attempts to map the relationship between varying input parameters and the resulting output structures, identifying required fields, optional headers, and authentication mechanisms.

Furthermore, generative AI can be used to synthesize formal documentation (like OpenAPI or Swagger specifications) directly from the inferred data, transforming raw network dumps into standardized, interactive API docs.

3. Detailed Student Activity Implementation

Phase 1 (Target Selection & Traffic Capture): Students will select a publicly available API with known documentation to serve as the ground truth. They will interact with it heavily, using tools like Postman, Wireshark, or Fiddler to capture a massive corpus of request-response pairs.

Phase 2 (Structure Analysis): Students will feed this raw traffic data into AI analysis tools. The AI will be tasked with identifying distinct endpoints, grouping similar requests, and deducing the JSON schema of the response payloads.

Phase 3 (Functionality Inference & Documentation Generation): Using prompt engineering or custom scripts, students will ask the AI to infer the business logic of the API (e.g., 'This endpoint likely creates a new user object'). The AI will then generate a complete OpenAPI specification.

Phase 4 (Comparative Validation): The critical final step involves a line-by-line comparison between the AI-generated documentation and the actual, official API specification.

4. Evaluation Metrics & Analytical Framework

The evaluation will focus on the completeness and structural accuracy of the recovered API documentation.

Metrics include the Endpoint Recovery Rate (percentage of total endpoints correctly identified), Parameter Accuracy (correct identification of data types like string vs. integer), and Schema Match Score (structural similarity between the generated JSON schema and the actual schema).

Students will also assess the AI's ability to correctly identify error codes and edge-case behaviors that were not explicitly present in the majority of the captured traffic.

5. Expected Outcomes & Real-World Impact

The expected outcome is a thorough assessment of the usefulness of AI for API understanding and automated documentation recovery.

Students will likely find that AI is highly effective at mapping standard RESTful structures and predicting basic CRUD (Create, Read, Update, Delete) operations based on HTTP verbs.

However, they will also document limitations, such as the AI's inability to guess undocumented security parameters, rate limits, or complex backend state changes that are not immediately visible in the network response.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness